

CSCE 606 SOFTWARE ENGINEERING · MODULE 05 · LECTURE NOTES

Testing

Buying confidence, and deciding how much to pay.

MODULE OUTCOMES

- Explain what tests actually buy and what they do not.
- Distinguish test sizes and their different purposes.
- Explain why coverage is a poor target.
- Write tests that survive refactoring.
- Assess the honest evidence on TDD.

1 What tests are actually for

CONFIDENCE TO CHANGE, NOT DEFECT DETECTION

The standard answer is that tests find bugs. True, and it is the secondary benefit.

The primary benefit is that a trustworthy test suite **makes change safe**. Without one, modifying unfamiliar code is a gamble, so people avoid it — they add new code beside the old rather than changing it, special-case around problems rather than fixing them, and eventually declare the system legacy and propose a rewrite.

A test suite is therefore not a quality gate attached at the end of development. It is the mechanism that prevents the cost of change from rising over time, which Module 01 identified as the entire point of the discipline.

Two secondary benefits are worth naming. Tests **document intent executably** — unlike comments, they cannot silently become false. And the requirement to be testable exerts **design pressure** toward lower coupling, because tightly coupled code is hard to test in isolation.

1.1 Sizing tests

The terms 'unit test' and 'integration test' generate more argument than clarity. Google's taxonomy sizes tests by *what they are permitted to touch*, which is objective and useful.

Size	May use	May not use	Typical runtime
Small	One process, one thread.	No file system, no network, no database, no sleeping, no real clock, no randomness.	Milliseconds. Run on every save.

Size	May use	May not use	Typical runtime
Medium	One machine; local database; multiple processes.	No external network.	Seconds. Run before every commit.
Large	Anything, including real external services.	—	Minutes. Run in CI, or before release.

The constraints make the categories enforceable — a test framework can prohibit network access in small tests — and they correlate directly with speed and with determinism, which are the properties that actually matter.

THE PYRAMID, AND ITS HONEST LIMITATION

The usual advice is many small tests, fewer medium, few large. The reasoning is sound: fast tests can be run constantly and a failure localises the problem immediately, whereas a large test failure says only that something somewhere broke.

The limitation is real: every small test can pass while the system is comprehensively broken, because the units work individually and do not fit together. Small tests verify the parts; only large tests verify the thing the user interacts with.

So the pyramid is guidance about *cost distribution*, not about value. Some end-to-end tests are mandatory, however slow and awkward, because nothing else tests what was actually promised. The right number is small and it is not zero.

2 Coverage

COVERAGE MEASURES EXECUTION, NOT VERIFICATION

A coverage tool reports that a line was *executed* during the test run. It cannot report whether anything about the result was checked.

A test suite that calls every function and asserts nothing achieves 100% line coverage and verifies nothing whatsoever. This is not a contrived example; suites written to satisfy a coverage mandate drift toward it naturally.

The asymmetry is what makes the metric usable at all. Low coverage is genuinely informative: code never executed by any test is definitely untested. High coverage is uninformative: it is consistent with excellent testing and with no assertions at all.

As a target it is actively harmful — a clean instance of Goodhart's law. Mandate 80% and the organisation produces exactly the tests that raise the number most cheaply, which are the least valuable tests in existence and which must then be maintained forever.

Better question	How to answer it
Do my tests actually verify anything?	Mutation testing. Automatically introduce small bugs — flip a comparison, change a constant — and measure how many the suite catches. Slow, and the only direct measure of test quality.
Can I refactor this safely?	Attempt a refactoring. If you cannot tell whether you broke something, the suite is inadequate whatever the coverage number says.
Does every bug fix include a regression test?	Check. This is a cheap, enforceable discipline that directly targets the code most likely to be wrong — code that has already been wrong once.
Which untested code would hurt most if it were wrong?	Rank by consequence, not by percentage. Payment handling without tests matters more than a logging helper without tests.

3 Tests that survive

TEST BEHAVIOUR, NOT IMPLEMENTATION

A test coupled to *how* code works fails whenever the code is restructured, even when the externally visible behaviour is identical.

This inverts the purpose entirely. The suite existed to make refactoring safe; now refactoring breaks the suite, so people avoid refactoring, so the design degrades. The tests have become a cost rather than an asset.

The usual symptom is heavy mocking with assertions about which internal methods were called, with which arguments, in which order. That is a test of the implementation, written in the vocabulary of the implementation.

Test through the public interface. If something is genuinely hard to test that way, that is useful information about the design — usually that a responsibility is in the wrong place or that a dependency should be injected.

```
# BRITTLE -- asserts how it works
def test_discount():
    repo, calc = Mock(), Mock()
    svc = OrderService(repo, calc)
    svc.apply_discount(order, "SAVE10")
    repo.find_by_code.assert_called_once_with("SAVE10")
    calc.compute.assert_called_once()

# DURABLE -- asserts what it does
def test_discount_reduces_total_by_ten_percent():
    order = Order(items=[Item(price=100)])
    result = apply_discount(order, discount_code("SAVE10"))
    assert result.total == 90
```

The second version is shorter, needs no mocking infrastructure, survives any internal restructuring, and — through its name — states the requirement. A reader learns what the system is supposed to do, which the first version never reveals.

Property	Requirement	Commonly broken by
Fast	Milliseconds for small tests.	Real I/O, sleeps, network calls, spinning up containers.
Deterministic	Identical result on every run.	Real clocks, real randomness, dependence on test ordering, concurrency.
Isolated	Passes regardless of what else ran.	Shared mutable fixtures, database state left behind, global configuration.
Readable	A failure message explains the problem without opening the test.	Bare assertions, obscure fixture setup, assertions on opaque values.
Focused	One reason to fail.	Twenty assertions, so the first failure hides the rest.

A FLAKY TEST IS WORSE THAN NO TEST

A test that fails intermittently trains everyone to re-run the build rather than investigate. Once that reflex is established, it applies to *every* failure, including real ones.

So one flaky test does not degrade the suite by one test's worth. It degrades trust in the whole suite, and a suite nobody trusts provides no confidence to change anything — which was the entire point.

The usual causes are real time, real randomness, dependence on test execution order, shared mutable state, real network calls, and assumptions about timing in concurrent code.

The response is to fix it or delete it, now. Quarantining flaky tests 'temporarily' is how suites die: the quarantine grows, nobody revisits it, and eventually most of the suite is not running.

4 TDD, assessed honestly

Test-driven development is usually presented as settled best practice. The evidence is weaker than the advocacy, and saying so is more useful than repeating the doctrine.

Claim	Support
Writing tests improves outcomes.	Well supported.
Writing a failing test before fixing a bug is valuable.	Well supported , and nearly free.
Designing for testability improves design.	Well supported. Code that is hard to test in isolation is usually too coupled.
Writing tests <i>first</i> , specifically, produces better code than writing them shortly after.	Not well supported. Studies conflict; effects, where found, are small and confounded with simply writing more tests.
Strict red-green-refactor measurably reduces defect rates.	Not well supported.

A DEFENSIBLE POSITION

Always write a failing test before fixing a bug. It proves the bug is real and reproducible, proves the fix works, and leaves a regression test behind. This costs almost nothing and is sensible regardless of what you think about TDD.

Write tests first when the interface is unclear. Writing the test forces you to be the first user of the API, which exposes awkwardness before you have built anything on top of it.

Write tests after when the design is obvious and you would rather explore in code first. This is a legitimate working style, not a moral failure.

Always write them. The ordering is a matter of working preference; whether the tests exist is not. Treating TDD as a tool with a domain of applicability, rather than as an identity, makes it more useful.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Software Engineering at Google — Chapters 11–14 (Testing Overview, Unit Testing, Test Doubles, Larger Testing)	The test-size taxonomy, the argument against mock-heavy testing, and the honest discussion of what large tests are for.
MIT 6.031 — Testing	Partition-based test design and systematic case selection, which this module leaves to Module 06.
Google Testing Blog — flaky tests and test sizes	Practical writing from people operating very large test suites.
Fucci et al. — An External Replication on the Effects of Test-driven Development (free)	One of the more careful empirical studies of TDD. Read it before accepting strong claims in either direction.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Take an untested piece of your own code and write tests for it. Record what you had to change in the code to make it testable, and what that tells you about its coupling.
- 2 Measure coverage on an existing project, then write a test that raises coverage substantially while asserting nothing. Confirm the number improves.
- 3 Run a mutation testing tool on the same project. Compare the mutation score against the coverage number and explain the difference.
- 4 Find a brittle test in a real codebase — one that asserts on mock call sequences — and rewrite it to assert behaviour. Then perform a refactoring and confirm the new test survives while the old one would not.
- 5 Deliberately introduce a flaky test using a real clock. Run the suite fifty times and record the failure rate. Then fix it by injecting the clock.
- 6 Take a bug from your own history. Write the failing test first, then fix it. Record how long each step took.
- 7 Classify an existing test suite by size using the constraint-based definition. Plot the distribution and compare it against the pyramid.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 What do tests primarily buy, and why is defect detection secondary?
- 2 How does the size taxonomy define small, medium, and large tests?
- 3 What is the pyramid's honest limitation, and what follows for end-to-end tests?
- 4 Why is high coverage uninformative while low coverage is informative?
- 5 Why is coverage harmful as a target?
- 6 What is the difference between testing behaviour and testing implementation, and how do you recognise the latter?
- 7 Why is a flaky test worse than no test at all?
- 8 Which claims about TDD are well supported and which are not?